

Stack Implementation in Java - idx vs top=-1 Methods

1. Introduction

A stack is a LIFO (Last In, First Out) data structure.

Two main approaches using arrays:

1. idx method (Next Free Position)
2. top = -1 method (Classic/Textbook)

2. idx Method (Next Free Position)

Concept: idx points to the next free position.

- Initial: `idx = 0`
- Top element: `arr[idx - 1]`
- Push: `arr[idx++] = value`
- Pop: `return arr[--idx]`
- Peek: `arr[idx - 1]`
- Empty: `idx == 0`
- Full: `idx == arr.length`

Memory Visualization:

Stack size = 4

`idx = 0` -> empty

push 10 -> `arr[0]=10`, `idx=1`

push 20 -> `arr[1]=20`, `idx=2`

push 30 -> `arr[2]=30`, `idx=3`

pop() -> `idx=2`, returns `arr[2]=30`

Pros: Clean, intuitive, tracks next free position.

Cons: Slightly less standard in textbooks.

3. top = -1 Method (Classic/Textbook)

Concept: top points to the current top element.

- Initial: `top = -1`
- Top element: `arr[top]`

- Push: `arr[++top] = value`
- Pop: `return arr[top--]`
- Peek: `arr[top]`
- Empty: `top == -1`
- Full: `top == arr.length - 1`

Memory Visualization:

Stack size = 4

Top = -1 -> empty

push 10 -> top=0, arr[0]=10

push 20 -> top=1, arr[1]=20

push 30 -> top=2, arr[2]=30

pop() -> return arr[2]=30, top=1

Pros: Standard, textbook style, widely recognized.

Cons: Slightly less intuitive for coding practice.

4. Side-by-Side Comparison

Feature	idx Method	top=-1 Method
Tracks	Next free index	Current top element
Initial	0	-1
Top element	<code>arr[idx-1]</code>	<code>arr[top]</code>
Push	<code>arr[idx++]</code>	<code>arr[++top]</code>
Pop	<code>arr[--idx]</code>	<code>arr[top--]</code>
Empty check	<code>idx == 0</code>	<code>top == -1</code>
Full check	<code>idx == arr.length</code>	<code>top == arr.length - 1</code>
Exam/Interview	safe	standard

5. Key Notes

- Both methods are correct.
- Use idx for coding practice.
- Use top=-1 for exams if preferred.
- Always handle overflow and underflow.
- arr can be final:

private final int[] arr;

- display() should print only valid stack elements.

6. Quick Tips for Exams

1. Draw stack after push/pop for clarity.
2. Remember difference: idx = next free, top=-1 = current top.
3. Overflow/Underflow checks are mandatory.
4. Clean, readable code always scores better.

7. Summary Diagram (Memory View)

idx Method:	top=-1 Method:
[10][20][30][_]	[10][20][30][_]
idx=3	top=2
Top element: arr[2]	Top element: arr[2]